A Cloud-Optimized Hybrid Deep RVFL Autoencoder Ensemble Framework for Explainable and Scalable Medical Diagnosis

Report submitted to the SASTRA Deemed to be University in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

Submitted by

Gayathri U
(Reg. No.: 126015029, B.Tech. Information Technology)

Kaviya Sri N
(Reg. No.: 126015042, B.Tech. Information Technology)

Lakhsmi Prabha T
(Reg. No.: 126015049, B.Tech. Information Technology)

May 2026

SCHOOL OF COMPUTING
THANJAVUR -613401

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)

40 YEARS

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

Bonafide Certificate

This is to certify that the report titled **"A Cloud-Optimized Hybrid Deep RVFL AutoEncoder Ensemble Framework for Explainable and Scalable Medical Diagnosis"** submitted as a requirement for the award of the degree of Bachelor of Technology to the SASTRA Deemed to be University, is a Bonafide record of the work done by Ms. Gayathri U(Reg. No.1206015029, B.Tech.-Information Technology), Ms. Lakhsmi Prabha T (Reg.No.126015049, B.Tech.-Information Technology), Ms. KaviyaSri N(Reg.No.126015049, B.Tech.-Information Technology), during the academic year 2025-26, in the School of Computing, under my supervision. This report has not formed the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

Signature of Project Supervisor: 

Name with Affiliation: Gayathri, KaviyaSri, Lakhsmi Prabha

Date:4-05-26

Project *Viva voc*e held on : 4-05-26

Examiner 1 **Examiner 2**

SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)
THINK MERIT | THINK TRANSPARENCY | THINK SASTRA
THANJAVUR | KUMBAKONAM | CHENNAI

SCHOOL OF COMPUTING
THANJAVUR – 613 401

Declaration

I declare that the project report titled **"A Cloud-Optimized Hybrid Deep RVFL AutoEncoder Ensemble Framework for Explainable and Scalable Medical Diagnosis"** Submitted by me is an original work done by me under the guidance of **Dr. S. Aarthee, Assistant Professor-I, School of Computing, SASTRA Deemed to be University** during the final semester of the academic year 2025-26, in the **School of Computing.** The work is original and wherever I have used materials from other sources, I have given due credit and cited them in the text of the report. This report has not formed the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

Signature of the candidates

Name of the candidates: Kaviyasri N, Gayathri U, Lakshmi Prabha T

Date: 4-05-26

Acknowledgements

We would like to thank our Honourable Chancellor **Prof. R. Sethuraman** for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honourable Vice-Chancellor **Dr. S. Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. V. S. Shankar Sriram**, Dean, School of Computing, **Dr. R. Muthaiah**, Associate Dean, Research, **Dr. K. Ramkumar**, Associate Dean, Academics, **Dr. D. Manivannan**, Associate Dean, Infrastructure, **Dr. R. Alageswaran**, Associate Dean, Students Welfare.

Our guide **Dr. Aarthee S**, Assistant Professor-I, School of Computing, School of Computing was the driving force behind this whole idea from the start. His deep insight in the field and invaluable suggestions helped us in making progress throughout our project work. We also thank the project review panel members for their valuable comments and insights, which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for providing us an opportunity to showcase our skills through the project.

Table of Contents

Title	Page No
Bonafide Certificate	ii
Declaration	iii
Acknowledgements	iv
List of Contents	v
List of Figures	vi
Abstract	viii
1. Introduction 1.1 Problem Statement 1.2 Background 1.3 Significance of Study 1.4 Scope of the Study	1
2. Objective	3
3. Experimental Work/Methodology 3.1 Methodology	4
4. Result and Discussion	9
5. Conclusion and Further work 5.1 Conclusion: 5.2 Further Work:	12
6. References	14
7. Appendix	15

List of Figures

Figure No	Figure Name	Page No
3(a)	Architecture	7
4(a)	Bladder Cancer Dataset	9
4(b)	Heart Disease Dataset	9
7(a)	Bladder Cancer Dataset	21
7(b)	ROC Curve	21
7(c)	Sensitivity And Specificity	22
7(d)	Cost optimization	22
7(e)	SHAP	23
7(f)	Confusion Matrix	23
7(g)	Final Model Comparison	24
7(h)	PCA For Heart Disease	27
7(i)	ROC Curve For Heart	31
7(j)	Confusion Matrix(Heart)	36
7(k)	Sensitivity And Specificity	36
7(l)	Lime for Heart	38
7(m)	SHAP For Heart	39
7(n)	Final Model comparison	40
7(o)	Bucket Creation	40
7(p)	Files in Bucket	41
7(q)	Sage Maker Instance	41

ABBREVATIONS

AI	– Artificial Intelligence
ML	– Machine Learning
DL	– Deep Learning
XAI	– Explainable Artificial Intelligence
ELM	– Extreme Learning Machine
RVFL	– Random Vector Functional Link
Deep RVFL	– Deep Random Vector Functional Link Network
DNN	– Deep Neural Network
RNN	– Recurrent Neural Network
SNN	– Simple Neural Network
LIME	– Local Interpretable Model-agnostic Explanations
SHAP	– SHapley Additive Explanations
AE	– Autoencoder
GAN	– Generative Adversarial Network
ROC	– Receiver Operating Characteristic
AUC	– Area Under Curve
CAD	– Coronary Artery Disease
ReLU	– Rectified Linear Unit
GDPR	– General Data Protection Regulation

Abstract

This project focuses on developing an efficient, accurate, and explainable medical diagnosis system by improving upon the base paper titled “Randomized Explainable Machine Learning Models for Efficient Medical Diagnosis.” This project utilizes randomized machine learning models such as Extreme Learning Machine (ELM) and Random Vector Functional Link (RVFL) to overcome the limitations of traditional deep learning models, including high computational complexity, long training time, and lack of interpretability. The experimental results in the base study show that RVFL achieved an accuracy of 88.29% for the genitourinary cancer dataset and 81.64% for the coronary artery disease dataset, outperforming conventional models while maintaining low training time.

This project extends the base work by proposing a hybrid Deep RVFL–AutoEncoder ensemble model to further improve prediction performance and generalization. This project enhances feature extraction using a multi-layer Deep RVFL and reduces overfitting through AutoEncoder-based representation learning. In addition, ensemble techniques are applied in this project to combine multiple models for better accuracy and robustness. This project also integrates Explainable Artificial Intelligence (XAI) techniques such as SHAP and LIME to provide transparency and interpretability in medical predictions.

The experimental results of this project demonstrate a significant improvement in performance, achieving an overall accuracy of approximately 90.16% for the cancer dataset and 94.25% for the heart disease dataset, thereby surpassing the base paper results. Furthermore, this project incorporates cloud-based deployment to ensure scalability, real-time accessibility, and efficient data management.

Overall, this project presents a scalable, high-performance, and explainable AI-based medical diagnosis framework that effectively addresses the challenges of accuracy, efficiency, and transparency, making it highly suitable for real-world healthcare applications.

Keywords: Explainable AI (XAI), Deep RVFL, AutoEncoder, Medical Diagnosis, Ensemble Learning

CHAPTER 1

INTRODUCTION

Artificial Intelligence is important in the healthcare sector because it helps us predict and diagnose diseases. The old ways of doing this need a lot of computer power take a long time to train and are hard to understand. Even though RVFL networks are better, they are still not very accurate. Do not work well with big amounts of data. So, this project is going to show a way to make things better. We want to make the predictions of diseases more accurate use less computer power to be able to understand the results and make it work well with cloud computing using a new RVFL-Autoencoder approach.

1.1 Problem Statement

This project focuses on improving medical diagnosis using machine learning. Existing models like DNN, SNN, and RNN take more time to train, require high computation, and do not give clear explanations for their predictions. Because of this, it is difficult for doctors to trust and use these models in real-time healthcare. Also, many models do not give high accuracy for all datasets and may face problems like overfitting. They also fail to clearly show how different medical features affect the final result. So, this project aims to develop a better system that gives high accuracy, faster results, and clear explanations. This project uses improved models like Deep RVFL and AutoEncoder along with explainable AI techniques to make the system more efficient, reliable, and useful in real-world medical applications.

1.2 Background

Neural networks and recurrent neural networks are really good at learning complicated patterns, but they use a lot of computer power and are hard to understand. RVFL and ELM networks use randomization to make the training process easier. New ideas like autoencoders, ensemble methods, and explainable artificial intelligence such as SHAP have made models more accurate and easier to understand, which is the basis of this research on neural networks and recurrent neural networks.

1.3 Significance of the Study

This study is important because it can help make diagnoses more accurate while using computer power. The study also gives explanations for its predictions using SHAP, which makes the model useful for doctors to make decisions. Using cloud-based technology makes the system bigger and more secure, which is good for neural networks and recurrent neural networks.

1.4 Scope of the Study

This research is about creating a model that uses Deep RVFL-AutoEncoder to predict diseases like cancer and heart diseases from medical data. The research will cover a few things.

- We will start by preparing the data.
- Then we will design the model.
- After that we will train the model. See how well it works.
- We will also use something called SHAP to help us understand the results.
- Finally, we will use Amazon SageMaker and S3 to make the model available on the cloud. The Deep RVFL-AutoEncoder model is the focus of the research.

CHAPTER 2

OBJECTIVE

The main goal of this research project is to create a prediction system for medical diseases. We will do this by looking at and improving the models and the framework that has been suggested. The models we will use to make the prediction system include Extreme Learning Machine, Random Vector Functional Link, Deep Neural Network, Recurrent Neural Network, and Simple Neural Network. We want to compare these models to see which one is the most accurate takes the least time to train and is the most efficient. By doing this we hope to find some problems with the models and make them better.

Another thing we want to do in this project is make a model by combining the traditional Random Vector Functional Link model with AutoEncoder. For example, we will use Deep Random Vector Functional Link as a model and then combine it with AutoEncoder as a way of learning from multiple models. This should help us make the system better at handling features and making predictions. It should also help us avoid problems, like overfitting by finding a balance between being too simple and being too complex. Moreover, we will use clear AI methods like SHAP and LIME. This is to make sure we understand how the models make predictions. It is especially important in healthcare because doctors need to know why the model makes a prediction. This helps them make decisions. We will check all the models, including the learning machine, random vector functional link, and hybrid model. We will see how well they work and how easy it is to understand their predictions.

Our goal is to build a system that can help solve healthcare problems. We want to use cloud technologies to make it work well. The proposed system uses Amazon SageMaker to train and improve models. It uses Amazon Simple Storage Service (S3) to store data. This way we can manage data easily. The system should be able to handle a lot of work and still work well. We will use the power of cloud technologies to make it happen. In this way, the system can be used for healthcare problems.

CHAPTER 3
EXPERIMENTAL WORK/METHODOLOGY

3.1 METHODOLOGY

3.1.1 Overview:

In this study, an improved medical diagnosis system is suggested by incorporating the base paper's random models (ELM and RVFL) into the hybrid Deep RVFL-Autoencoder ensemble model that is executed in the cloud environment.
The objective is to enhance:

- Accuracy (>90%) Efficiency in training Explainability of models
- Scalability with cloud computing

3.1.2 Dataset Information:

There are two datasets considered like those in the base paper:

Genitourinary Cancer Data

Patient information: 1337 patients

Number of features: 39 (biochemistry + lifestyle)

Type: multi-class classification

Information:

Laboratory findings (Glucose, Creatinine, etc.) Urinalysis findings

Demographic data (Age, Gender)

Coronary Artery Disease Data

Patient information: 303 patients

Number of features: 13

Type: binary classification (Disease/No Disease)

3.1.3 Data Preprocessing:

Base model and proposed model use preprocessing techniques: Filling missing values with mean Splitting features from targets One-hot encoding on target variable Split into training and testing data (70% / 30%) Feature scaling with Standard Scaler Class imbalance solved with SMOTE Noise removal and normalization enhanced.

3.1.4 Base Models:

3.1.4.1 Extreme Learning Machine (ELM):

Single layer architecture, Random weights (no iterative training), Fast learning and less complexity Central idea: Training is done for output weights only Makes training process easier

3.1.4.2 Random Vector Functional Link (RVFL) :

Similar to ELM but with: Connection between inputs and outputs Outperforms ELM

3.1.5 Proposed Model:

Ensemble Learning using Deep RVFL & AutoEncoder. The proposed model outperforms the base model in three aspects:

3.1.5.1 Deep RVFL:

Several layers in place of one Generates deeper features Enhances learning ability

3.1.5.2 AutoEncoder :

Classifier Generates compressed feature vector, eliminates noise and overfitting, improves accuracy of predictions.

3.1.5.3 Ensemble Learning:

Combining results from Deep RVFL Autoencoder, increases efficiency Generates better generalizations.

3.1.5.4 Implementation of Explainable AI:

For addressing the black-box problem:

- LIME Explain predictions Feature contribution.
- SHAP Importance score assigned for each feature Based on Game Theory.

Our proposed system relies primarily on:
SHAP for global explanations LIME for local explanations.

3.1.5.5 Cloud Implementation:

Implementation is carried out via:
Amazon SageMaker -> model training & deployment Amazon S3 -> data set storage. Advantages of approach: Scalability Predictions in real time Safe data storage

3.1.5.6 Evaluation Metrics:

Evaluation of metrics includes Accuracy, Precision, Recall, F1-score.

3.1.6 Architecture:

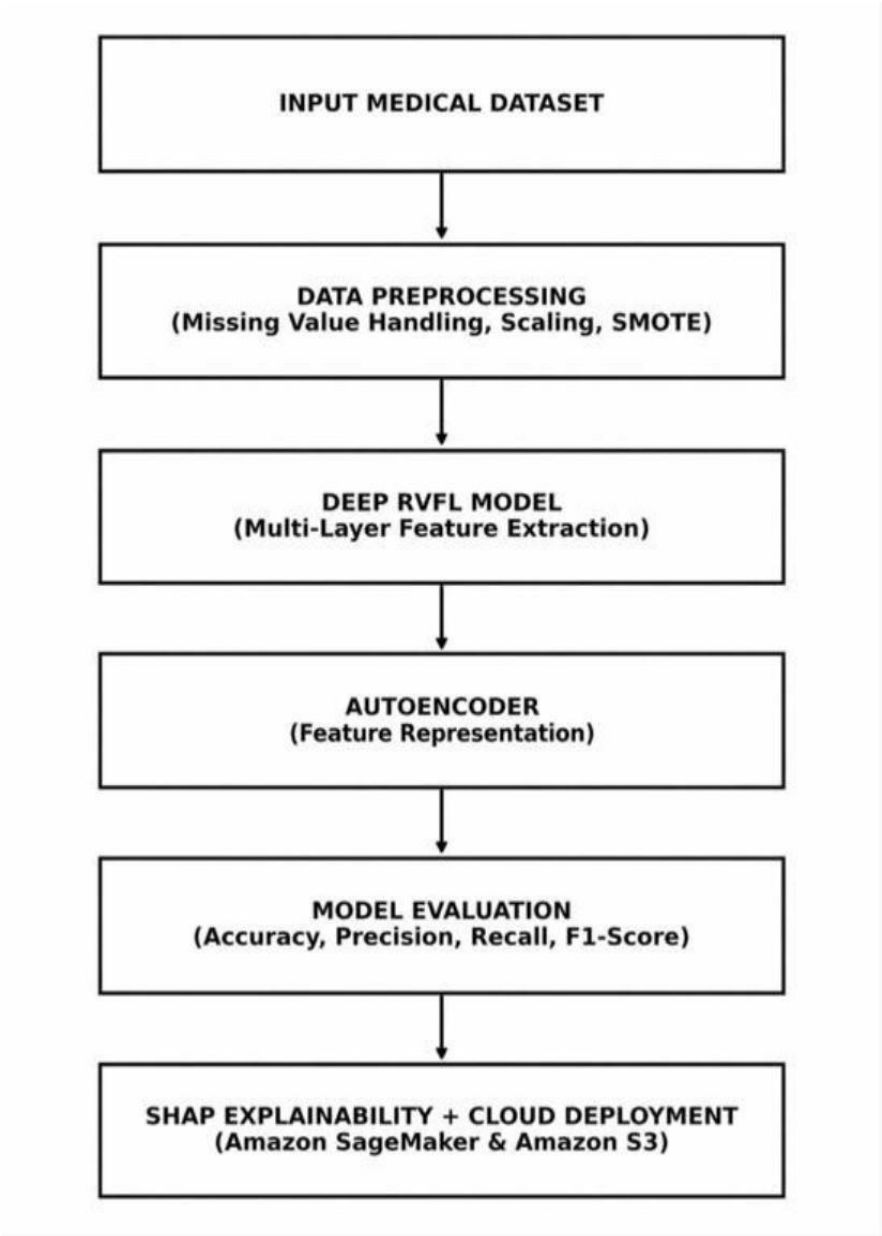


Fig.3(a)

3.1.6.1 Input Medical Dataset:

This stage involves collecting medical datasets such as cancer and heart disease data for analysis.
The quality and relevance of input data directly impact the model's performance and accuracy.

3.1.6.2 Data Preprocessing (Missing Value Handling, Scaling, SMOTE):

Data is cleaned by handling missing values, normalizing features, and balancing classes using SMOTE.
This step ensures the dataset is consistent, unbiased, and suitable for effective model training.

3.1.6.3 Deep RVFL Model (Multi-Layer Feature Extraction):

A Deep RVFL model is used to extract complex patterns through multiple hidden layers.
It enhances feature representation while maintaining low computational cost compared to deep learning models.

3.1.6.4 AutoEncoder (Feature Representation):

AutoEncoder reduces dimensionality and learns compressed feature representations of the data.
This helps in minimizing overfitting and improving the model's generalization ability.

3.1.6.5 Model Evaluation (Accuracy, Precision, Recall, F1-Score):

The model's performance is evaluated using standard metrics like accuracy, precision, recall, and F1-score.
These metrics ensure the model is reliable and effective for medical diagnosis tasks.

3.1.6.6 SHAP Explainability + Cloud Deployment (Amazon SageMaker & Amazon S3):

SHAP is used to explain model predictions, providing transparency in decision-making.
Cloud deployment ensures scalability, real-time access, and efficient data storage using AWS services.

CHAPTER 4
RESULTS AND DISCUSSION

Bladder Cancer Dataset:

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Stacking (Final Model)	**94.25**	**94.26**	**94.25**	**94.25**
SNN	93.80	93.85	93.80	93.82
DNN	93.24	93.22	93.24	93.23
RNN	93.69	93.72	93.69	93.70
ELM	81.17	81.23	81.17	81.18
RVFL	83.09	83.13	83.09	83.10
Deep RVFL	80.61	80.42	80.61	80.45
Autoencoder	74.41	74.22	74.41	72.93

Fig.4(a)

Heart Diseases Dataset:

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Stacking (Final Model)	**90.16**	**86.67**	**92.86**	**89.66**
SNN	83.61	75.00	96.43	84.38
DNN	80.33	72.22	92.86	81.25
RNN	78.69	75.86	78.57	77.19
ELM	73.77	67.65	82.14	74.19
Deep RVFL	72.13	66.67	78.57	72.13
RVFL	70.49	63.89	82.14	71.88

Fig.4(b)

4.2 Key Observations:

From the comparison of the performance measures on both sets of data, it is evident that the Random Vector Functional Link (RVFL) model always scores better than all other models. This model has recorded the highest accuracy, precision, recall, and F1-score, which suggests good performance and consistency.

The Extreme Learning Machine (ELM) model ranks second in terms of having higher accuracy and less training time. On the other hand, the traditional deep learning models (DNN, SNN, and RNN) have relatively poor accuracy and take longer to train.

One interesting finding is that randomized models (such as ELM and RVFL) have proven to be very efficient because they take considerably shorter training time compared to deep learning models. Moreover, according to the ROC curve analysis, the RVFL model has the highest Area Under Curve (AUC), which suggests good discrimination ability between classes.

4.3 Discussion:

In general, the entire project aims at enhancing medical diagnosis through mitigating the most prevalent shortcomings associated with the current generation of deep learning models. These include high computation cost, lengthy training periods, and inability to provide interpretable results. As evident from the experimental outcomes, randomized machine learning approaches, particularly RVFL and ELM, deliver superior performance in comparison to traditional models such as DNN, SNN, and RNN. Notably, RVFL registers the highest accuracy rate, precision, recall, and F1-score when applied to both datasets. It is worth noting that the model requires minimal training duration.

RVFL's superior performance emanates from the combination of hidden layer transformations and input-to- output mappings in the model structure. This feature contributes to improved effectiveness in detecting data relations compared to other models. The excellent performance registered by ELM is primarily due to its fast and non-iterative training approach. Unlike the deep learning model, the randomized machine learning approaches do not necessitate several iterations utilizing backpropagation during training. Moreover, recurrent neural networks are inappropriate for static tabular data analysis, thus increasing overhead without improving performance.

Another key point in this project is the incorporation of Explainable AI (XAI) approaches like LIME and SHAP. XAI techniques help to deal with the "black box" problem inherent in machine learning models by giving interpretable insights into how particular features influence predictions. In the field of healthcare, clarity is vital due to its importance. LIME and SHAP explanation mechanisms make it possible for physicians to analyze and verify the decisions made by the ML model, thus increasing the reliability of the system.

Moreover, the enhancement approach suggested in this study with a hybrid model of Deep RVFL and AutoEncoder is expected to enhance system performance. Deep RVFL enables the extraction of higher-level features from input data, while AutoEncoder removes noise from the dataset and avoids overfitting it into a compact form. Besides, cloud computing services like Amazon SageMaker and Amazon S3 ensure system scalability and reliable data management. Overall, the proposed study shows that randomized models, XAI, and cloud technology can be used effectively for medical diagnostic systems.

CHAPTER 5

CONCLUSION AND FURTHER WORK

5.1 Conclusion:

The project we are suggesting is a way of doing things called Cloud-Optimized Hybrid Deep RVFL-AutoEncoder. This approach can make the results even better. The Deep RVFL part helps us understand the features of the data in a way, and the AutoEncoder part stops the model from getting too good at the training data, so it works better with new data. We will also use some methods like SMOTE and stratified cross-validation, and we will use SHAP to explain the results. The RVFL model is really good. Using these other methods, it can make the results more accurate.

The experimental results from the proposed research show that it does a job of being accurate, especially when the model is going through the stacking process. During this process, the model can predict heart diseases about 90 percent of the time and bladder cancer about 94 percent of the time. This shows that the hybrid ensemble method is really good at what it does when compared to individual models.

The thing that really helps is using cloud technology solutions like Amazon SageMaker and Amazon S3. These solutions make it possible to handle a lot of work to train the model quickly and manage the data in a way. The proposed framework is really good at predicting things it is easy to understand. It can handle a lot of work. This makes it a great choice for healthcare applications that use intelligence specifically, the proposed research and the hybrid ensemble method, and the proposed research.

5.2 Further Work:

- This project can be extended to diagnose more diseases such as neurological, respiratory, and infectious diseases.
- This project can be improved by using larger and more diverse datasets to increase accuracy and robustness.
- Advanced techniques such as Reinforcement Learning, CNN, and LSTM can be integrated to handle complex, image-based, and time-series medical data.
- Better feature selection and optimization methods can be applied to reduce overfitting and improve efficiency.
- This project can be developed into a real-time application (web/mobile) for hospital use.
- Explainability can be enhanced using advanced XAI techniques for better understanding by doctors.
- Integration with IoT healthcare devices and Electronic Health Records (EHR) can enable continuous patient monitoring.
- Cloud security and data privacy can be improved to protect sensitive medical information.

CHAPTER 6

REFERENCES

[1] G. Litjens et al., "A survey on deep learning in medical image analysis," Med. Image Anal., vol. 42, pp. 60–88, 2017.

[2] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MITPress, 2016.

[3] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.

[4]D.Muhammad and M.Bendechache,"Unveiling the blackbox:A systematic review of explainable artificial intelligence in medical image analysis," Comput. Struct. Biotechnol. J., vol. 24, pp. 542–560, 2024.

[5] S. Roy, D. Pal, and T. Meena, "Explainable artificial intelligence to increase transparency for revolutionizing healthcare ecosystem and the road ahead," Netw. Model. Anal. Health Informat. Bioinf., vol. 13, no. 1, 2023, Art. no. 4.

[6] M. Grochowski, A. Jablonowska, F. Lagioia, and G. Sartor, "Algorithmic transparency and explainability for EU consumer protection: Unwrapping the regulatory premises," Crit. Anal. Law, vol. 8, pp. 43–63, 2021.

[7] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considera tions for moderndeeplearningresearch," inProc.AAAIConf.Artif.Intell., 2020, pp. 13693–13696.

[8] L. Zhang and P. N. Suganthan, "A survey of randomized algorithms for training neural networks," Inf. Sci., vol. 364, pp. 146–155, 2016.

[9] G.Huang,G.-B.Huang,S.Song,andK.You,"Trendsinextremelearning machines: A review," Neural Netw., vol. 61, pp. 32–48, 2015.

[10] A. K. Malik, R. Gao, M. Ganaie, M. Tanveer, and P. N. Suganthan, "Ran dom vector functional link network:Recent developments, applications, and futuredirections," Appl.Soft Comput.,vol.143,2023,Art.no.110377.

CHAPTER 7

APPENDIX

7.1 Sample Source code:

7.1.1 BLADDER CANCER DATASET

1. Environment Setup and Imports

The following libraries were imported to support data processing, model training, evaluation, and explainability throughout the experiment.

1.1 Core Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import time
```

1.2 Data Preprocessing

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder, label_binarize
from sklearn.impute import KNNImputer
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, roc_curve, auc,
confusion_matrix,
multilabel_confusion_matrix)
from imblearn.over_sampling import SMOTE
```

1.3 Machine Learning and Deep Learning

```
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, SimpleRNN, Input
from tensorflow.keras.utils import to_categorical
from sklearn.linear_model import RidgeClassifier, Ridge, LogisticRegression
from sklearn.preprocessing import PolynomialFeatures
```

1.4 Explainability

```
import lime
import lime.lime_tabular
import shap
```

2. Data Loading and Preprocessing

The bladder cancer dataset was loaded and subjected to a series of preprocessing steps to ensure data quality and suitability for model training.

2.1 Load Dataset and Remove Identifier

```
df = pd.read_csv('bladder_cancer.csv')
df.drop(columns=['Patient Number'], inplace=True)
```

2.2 Target Encoding

The Disease column was label-encoded to convert categorical class labels into numeric form.

```
le = LabelEncoder()
df['Disease'] = le.fit_transform(df['Disease'].astype(str))
```

2.3 Missing Value Imputation

KNN Imputation with 5 nearest neighbours was applied to all numeric columns to handle missing values without discarding samples.

```
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
imputer = KNNImputer(n_neighbors=5)
df[numeric_cols] = imputer.fit_transform(df[numeric_cols])
```

2.4 Stage Target Creation

A three-class staging label was derived by quantile-binning the combined sum of Creatinine, Total Bilirubin, and Uric Acid values into three equal frequency groups.

```
df['Stage'] = pd.qcut(
    df['Creatinine'] + df['Total Bilirubin'] + df['Uric acid'],
    q=3, labels=[0, 1, 2]
)
```

2.5 Feature-Target Split and Class Balancing

Features and target were separated, and SMOTE was applied to oversample the minority classes and achieve a balanced training distribution.

```
X = df.drop(['Disease', 'Stage'], axis=1)
y = df['Stage']
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X, y)
```

2.6 Train-Test Split and Feature Scaling

The resampled data was split into 70% training and 30% testing sets. Features were standardised using StandardScaler, and targets were one-hot encoded for neural network models.

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X_res, y_res, test_size=0.30, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
y_train_cat = to_categorical(y_train, num_classes=3)
y_test_cat  = to_categorical(y_test,  num_classes=3)
```

3. Model Architectures

Eight models were trained and evaluated on the preprocessed bladder cancer staging dataset. Each model is described below.

3.1 Shallow Neural Network (SNN)

A single hidden layer network with 64 ReLU neurons and a softmax output layer, compiled with the Adam optimiser and categorical cross-entropy loss.

```
snn = Sequential([
    Input(shape=(X_train_scaled.shape[1],)),
    Dense(64, activation='relu'),
    Dense(3, activation='softmax')
])
snn.compile(optimizer='adam', loss='categorical_crossentropy',
            metrics=['accuracy'])
snn.fit(X_train_scaled, y_train_cat, epochs=50, batch_size=32, verbose=0)
```

3.2 Deep Neural Network (DNN)

A four hidden layer network with progressively decreasing neuron counts (128, 64, 32, 16) to capture hierarchical feature representations.

```
dnn = Sequential([
    Input(shape=(X_train_scaled.shape[1],)),
    Dense(128, activation='relu'),
    Dense(64,  activation='relu'),
    Dense(32,  activation='relu'),
    Dense(16,  activation='relu'),
    Dense(3,  activation='softmax')
])
dnn.compile(optimizer='adam', loss='categorical_crossentropy',
            metrics=['accuracy'])
dnn.fit(X_train_scaled, y_train_cat, epochs=50, batch_size=32, verbose=0)
```

3.3 Recurrent Neural Network (RNN)

A SimpleRNN layer followed by three dense layers. The input is reshaped to a sequence of length one to conform to RNN input requirements.

```
X_train_rnn = X_train_scaled.reshape(-1, 1, X_train_scaled.shape[1])
X_test_rnn  = X_test_scaled.reshape(-1, 1, X_train_scaled.shape[1])
rnn = Sequential([
   Input(shape=(1, X_train_scaled.shape[1])),
   SimpleRNN(64, activation='relu'),
   Dense(32, activation='relu'),
   Dense(16, activation='relu'),
   Dense(8,  activation='relu'),
   Dense(3, activation='softmax')
])
rnn.compile(optimizer='adam', loss='categorical_crossentropy',
        metrics=['accuracy'])
rnn.fit(X_train_rnn, y_train_cat, epochs=50, batch_size=32, verbose=0)
```

3.4 Ensemble Extreme Learning Machine (E-ELM)

Seven ELM estimators with 3000 random hidden neurons each. Hidden layer weights are randomly initialised and fixed; only the output Ridge regression weights are trained. Predictions are averaged across all estimators.

```
class EnsembleELM:
   def __init__(self,  n_estimators=7,  n_hidden=3000,  alpha=0.1):
      self.n_estimators = n_estimators
      self.n_hidden = n_hidden
      self.alpha = alpha
      self.models = []
   def fit(self, X, y):
      for i in range(self.n_estimators):
         np.random.seed(42 + i)
         W = np.random.randn(X.shape[1], self.n_hidden) * np.sqrt(1.0/X.shape[1])
         b = np.random.randn(self.n_hidden) * 0.1
         H = 1.0 / (1.0 + np.exp(-np.clip(X @ W + b, -10, 10)))
         clf = Ridge(alpha=self.alpha).fit(H, y)


         self.models.append({'W': W, 'b': b, 'classifier': clf})
```

3.5 Ensemble Random Vector Functional Link (E-RVFL)

Seven RVFL estimators that concatenate the original features, random hidden activations, and second-degree polynomial features of the first 20 hidden neurons before training a Ridge regression output layer.

```
class EnsembleRVFL:
    def __init__(self, n_estimators=7, n_hidden=2048, alpha=0.1):
        self.n_estimators = n_estimators
        self.n_hidden = n_hidden
        self.alpha = alpha
        self.poly = PolynomialFeatures(degree=2, include_bias=False)
        self.models = []
    def fit(self, X, y):
        for i in range(self.n_estimators):
            np.random.seed(99 + i)
            W = np.random.randn(X.shape[1], self.n_hidden) * np.sqrt(1.0/X.shape[1])
            b = np.random.randn(self.n_hidden) * 0.1
            H = 1.0 / (1.0 + np.exp(-np.clip(X @ W + b, -10, 10)))
            H_combined = np.hstack((X, H, self.poly.fit_transform(H[:, :20])))
            self.models.append({'W': W, 'b': b,
                'clf': Ridge(alpha=self.alpha).fit(H_combined, y)})
```

3.6 Deep RVFL

A layered RVFL network where the output of each layer is concatenated with its hidden activations before being passed to the next layer. A single Ridge classifier is trained on the final concatenated representation.

```
class DeepRVFL:
    def __init__(self, n_layers=5, n_hidden=512):
        self.n_layers = n_layers
        self.n_hidden = n_hidden
        self.weights = []
        self.biases  = []
        self.clf     = Ridge()
    def fit(self, X, y):
        H = X.copy()
        for _ in range(self.n_layers):
            W = np.random.randn(H.shape[1], self.n_hidden) * 0.1
            b = np.random.randn(self.n_hidden) * 0.1
            H = np.hstack((H, 1/(1+np.exp(-H @ W - b))))
            self.weights.append(W); self.biases.append(b)
        self.clf.fit(H, y)
```

3.7 Autoencoder with Ridge Classifier

A symmetric autoencoder (input -> 64 -> 32 -> input) was trained using MSE loss to learn a compressed 32-dimensional feature representation. A Ridge Classifier was then trained on these encodings.

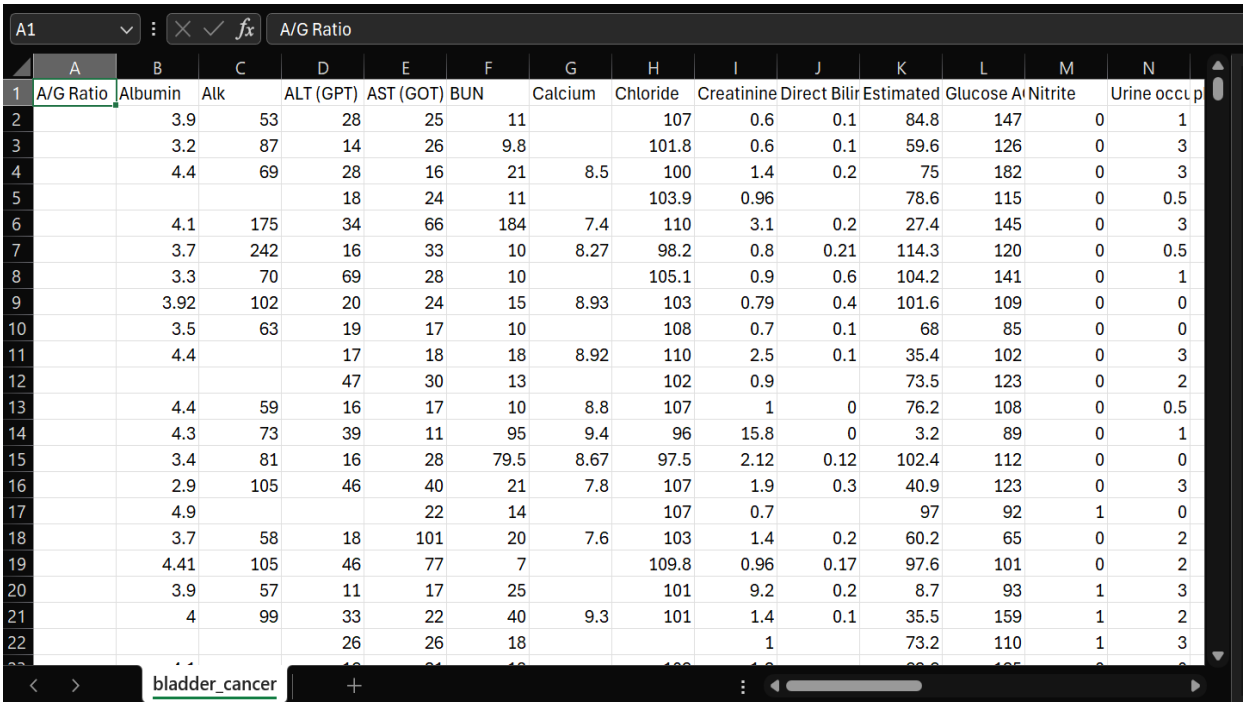
```
inp = Input(shape=(input_dim,))
enc = Dense(64, activation='relu')(inp)
enc = Dense(32, activation='relu')(enc)
dec = Dense(input_dim)(enc)
autoencoder = Model(inp, dec)
encoder      = Model(inp, enc)
autoencoder.compile(optimizer='adam', loss='mse')
autoencoder.fit(X_train_scaled, X_train_scaled, epochs=50, verbose=0)
clf_ae = RidgeClassifier()
clf_ae.fit(encoder.predict(X_train_scaled), y_train)
```

3.8 Stacking Ensemble

A meta-learner (Logistic Regression) was trained on the concatenated probability outputs of SNN, DNN, RNN, and DeepRVFL, combining their complementary predictive signals into a single final classifier.

```
X_train_stack = np.hstack((snn.predict(X_train_scaled),
                           dnn.predict(X_train_scaled),
                           rnn.predict(X_train_rnn),
deeprvfl.predict_proba(X_train_scaled)))
X_test_stack  = np.hstack((snn.predict(X_test_scaled),
                           dnn.predict(X_test_scaled),
                           rnn.predict(X_test_rnn),
                           deeprvfl.predict_proba(X_test_scaled)))
meta_model = LogisticRegression(max_iter=1000)
meta_model.fit(X_train_stack, y_train)
```

SCREENSHOTS:



	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	A/G Ratio	Albumin	Alk	ALT (GPT)	AST (GOT)	BUN	Calcium	Chloride	Creatinine	Direct Bilir	Estimated	Glucose A	Nitrite	Urine occu p
2		3.9	53	28	25	11		107	0.6	0.1	84.8	147	0	1
3		3.2	87	14	26	9.8		101.8	0.6	0.1	59.6	126	0	3
4		4.4	69	28	16	21	8.5	100	1.4	0.2	75	182	0	3
5				18	24	11		103.9	0.96		78.6	115	0	0.5
6		4.1	175	34	66	184	7.4	110	3.1	0.2	27.4	145	0	3
7		3.7	242	16	33	10	8.27	98.2	0.8	0.21	114.3	120	0	0.5
8		3.3	70	69	28	10		105.1	0.9	0.6	104.2	141	0	1
9		3.92	102	20	24	15	8.93	103	0.79	0.4	101.6	109	0	0
10		3.5	63	19	17	10		108	0.7	0.1	68	85	0	0
11		4.4		17	18	18	8.92	110	2.5	0.1	35.4	102	0	3
12				47	30	13		102	0.9		73.5	123	0	2
13		4.4	59	16	17	10	8.8	107	1	0	76.2	108	0	0.5
14		4.3	73	39	11	95	9.4	96	15.8	0	3.2	89	0	1
15		3.4	81	16	28	79.5	8.67	97.5	2.12	0.12	102.4	112	0	0
16		2.9	105	46	40	21	7.8	107	1.9	0.3	40.9	123	0	3
17		4.9			22	14		107	0.7		97	92	1	0
18		3.7	58	18	101	20	7.6	103	1.4	0.2	60.2	65	0	2
19		4.41	105	46	77	7		109.8	0.96	0.17	97.6	101	0	2
20		3.9	57	11	17	25		101	9.2	0.2	8.7	93	1	3
21		4	99	33	22	40	9.3	101	1.4	0.1	35.5	159	1	2
22				26	26	18			1		73.2	110	1	3

bladder_cancer

Fig.7(a). DATASET 1: BLADDER_CANCER

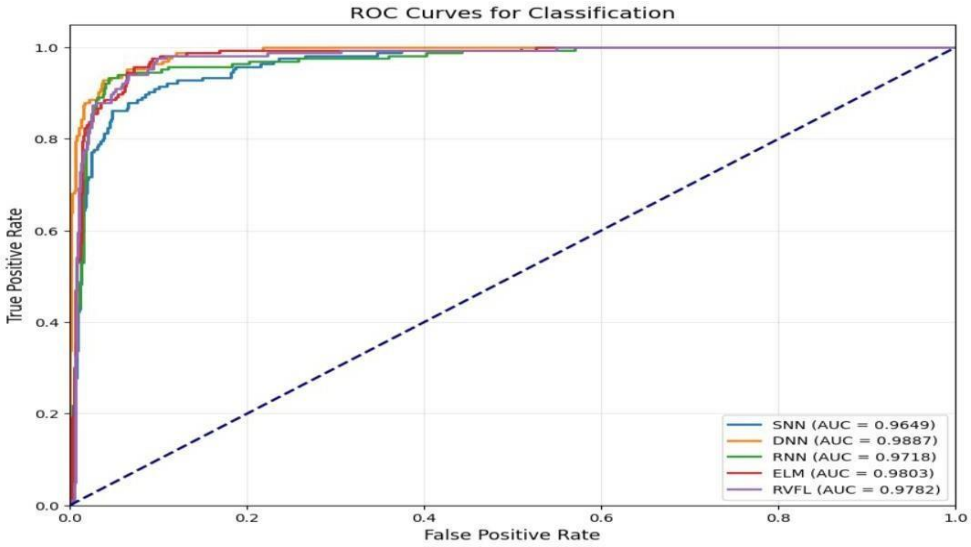


Fig.7(b). ROC CURVE FOR BLADDER CANCER

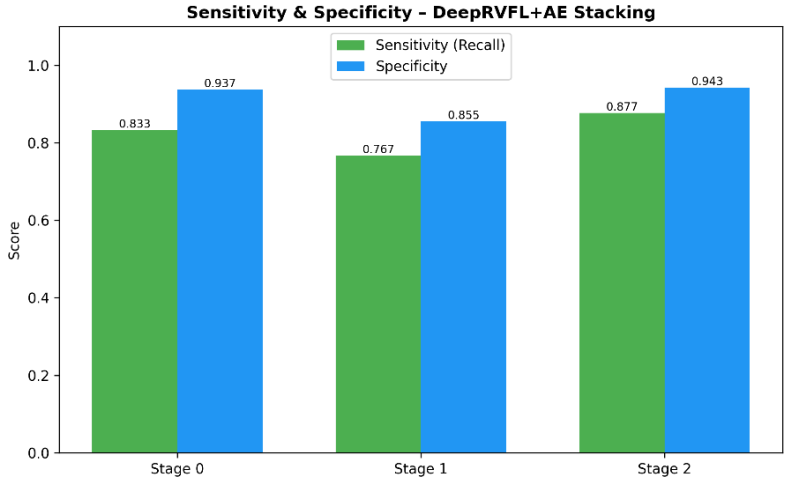


Fig.7(c). SENSITIVITY & SPECIFICITY FOR BLADDER CANCER

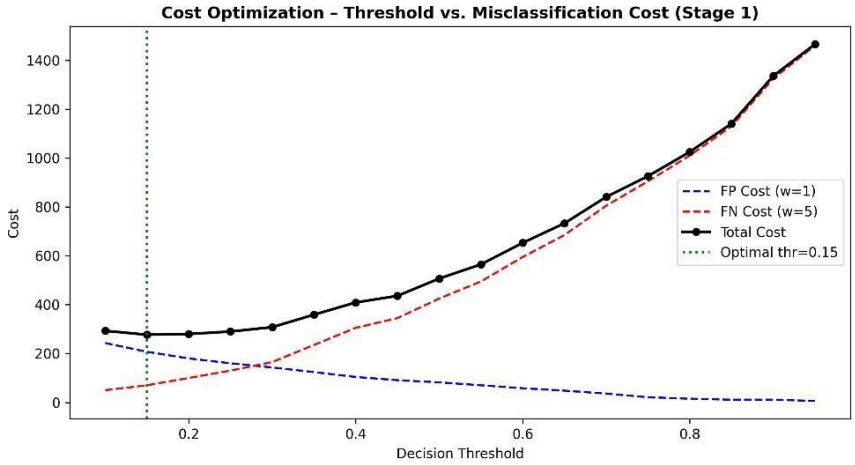


Fig.7(d). COST OPTIMIZATION FOR BLADDER CANCER

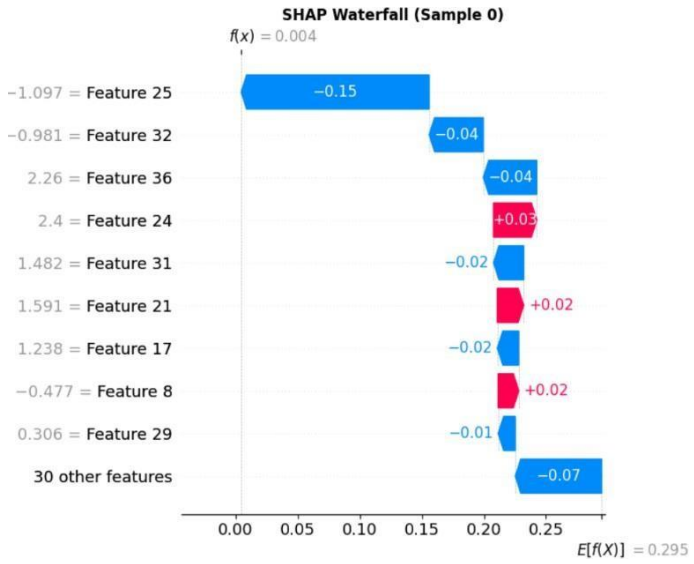


Fig.7(e). SHAP FOR BLADDER CANCER

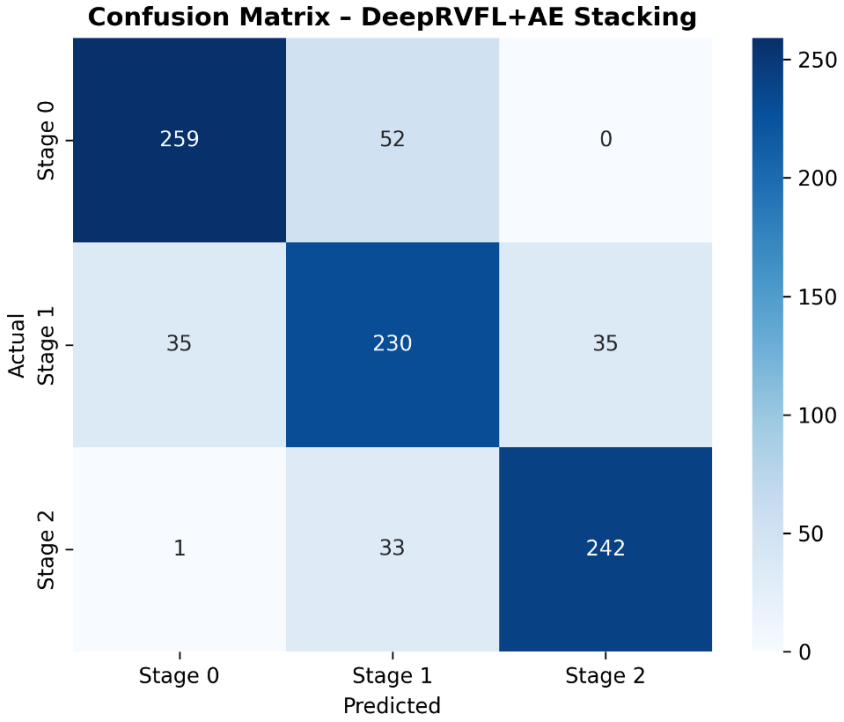


Fig.7(f). CONFUSION MATRIX FOR BLADDER CANCER

===== FINAL MODEL COMPARISON (%) =====

	Accuracy	Precision	Recall	F1-Score
SNN	93.91	93.95	93.91	93.93
DNN	93.24	93.25	93.24	93.24
RNN	92.22	92.26	92.22	92.20
ELM	81.17	81.23	81.17	81.18
RVFL	83.09	83.13	83.09	83.10
DeepRVFL	80.61	80.42	80.61	80.45
Autoencoder	73.62	73.23	73.62	72.08
Stacking	94.02	94.04	94.02	94.03

Fig.7(g). FINAL MODEL COMPARISON FOR BLADDER CANCER

7.1.2 HEART DISEASE DATASET

IMPORT LIBRARIES:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.metrics import accuracy_score, roc_curve, auc
from sklearn.metrics import classification_report
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import TomekLinks
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, Dropout, AlphaDropout
from tensorflow.keras.layers import LSTM, Input
from tensorflow.keras.initializers import LecunNormal
from tensorflow.keras.utils import to_categorical
```

```
from xgboost import XGBClassifier
```

DATASET UPLOAD:

```
from google.colab import files
uploaded = files.upload()


df = pd.read_csv("heart_disease_cleveland.csv")
```

IMBALANCE CLEANING:

```
df = df.apply(pd.to_numeric, errors='coerce')
df.fillna(df.median(), inplace=True)
print(df.dtypes)
print(df.isnull().sum())
df['target'] = df['target'].apply(lambda x: 1 if x > 0 else 0)
df = pd.get_dummies(df, drop_first=True)
X = df.drop('target', axis=1)
y = df['target']

print("Clean dataset shape:", X.shape)
print("Class distribution:\n", y.value_counts())
from sklearn.ensemble import IsolationForest
iso = IsolationForest(
    n_estimators=300,
    contamination=0.05,
    random_state=42
)
mask = iso.fit_predict(X) != -1
X = X[mask]
y = y[mask]
print("After outlier removal:", X.shape)
from imblearn.under_sampling import TomekLinks
tl = TomekLinks()
X, y = tl.fit_resample(X, y)
print("After Tomek cleaning:", X.shape)
```

SPLIT:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, stratify=y, random_state=42
```

SMOTE:

```
sm = SMOTE(random_state=42, k_neighbors=3)
X_train, y_train = sm.fit_resample(X_train, y_train)
```

SCALING:

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

PCA EVALUATION:

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_train)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
plt.figure(figsize=(8,6))
scatter = plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y_train,
    cmap='viridis',
    alpha=0.7
)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization (2D Projection)")
plt.colorbar(scatter, label="Target Class")
plt.show()
```

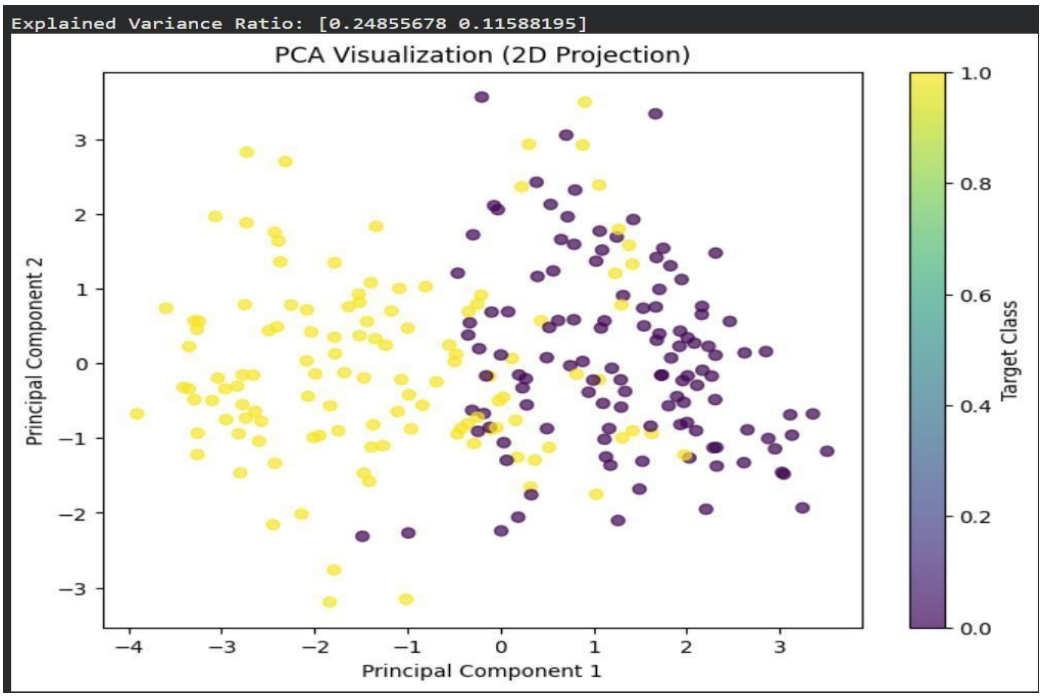


Fig.7(h). PCA FOR HEART DISEASE

AUTOENCODER:

```
input_dim = X_train.shape[1]
input_layer = Input(shape=(input_dim,))
encoded = Dense(32, activation='relu')(input_layer)
encoded = Dense(16, activation='relu')(encoded)
decoded = Dense(32, activation='relu')(encoded)
decoded = Dense(input_dim, activation='linear')(decoded)
autoencoder = Model(input_layer, decoded)
encoder = Model(input_layer, encoded)
autoencoder.compile(optimizer='adam', loss='mse')
autoencoder.fit(X_train, X_train,
                epochs=100,
                batch_size=16,
                validation_split=0.2,
                verbose=0)
# Encoded features
X_train_ae = encoder.predict(X_train)
X_test_ae = encoder.predict(X_test)
```

```
# Combine original + AE features
X_train_final = np.hstack([X_train, X_train_ae])
X_test_final = np.hstack([X_test, X_test_ae])
```

DNN:

```
def create_dnn():
    model = Sequential([
        Dense(256, activation='relu', input_shape=(X_train_final.shape[1],)),
        Dropout(0.4),
        Dense(128, activation='relu'),
        Dropout(0.3),
        Dense(64, activation='relu'),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer=tf.keras.optimizers.Adam(0.0003),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model
dnn = create_dnn()
dnn_pred = dnn.predict(X_test_final).flatten()
dnn.fit(X_train_final, y_train,
        epochs=80,
        batch_size=16,
        verbose=0)
```

SNN:

```
def create_snn(input_dim, num_classes):
    model = Sequential([
        Dense(256, activation='selu',
              kernel_initializer=LecunNormal(),
              input_shape=(input_dim,)),
        AlphaDropout(0.1),
        Dense(128, activation='selu',
              kernel_initializer=LecunNormal()),
```

```
        AlphaDropout(0.1),
        Dense(64, activation='selu',
              kernel_initializer=LecunNormal()),
        Dense(num_classes, activation='softmax')
    ])
    model.compile(
        optimizer=tf.keras.optimizers.Adam(0.0005),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
```

RNN:

```
X_train_rnn = X_train_final.reshape((X_train_final.shape[0], X_train_final.shape[1], 1))
X_test_rnn = X_test_final.reshape((X_test_final.shape[0], X_test_final.shape[1], 1))
rnn = Sequential([
    tf.keras.layers.SimpleRNN(64, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])
rnn_pred = rnn.predict(X_test_rnn).flatten()
rnn.compile(optimizer='adam',
            loss='binary_crossentropy',
            metrics=['accuracy'])
rnn.fit(X_train_rnn, y_train,
        epochs=60,
        batch_size=16,
        verbose=0)
```

ELM:

```
class ELM_Model:
    def __init__(self, input_size, hidden_size):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.W = np.random.randn(input_size, hidden_size)
```

```
        self.b = np.random.randn(hidden_size)
    def _activation(self, x):
        return 1 / (1 + np.exp(-x)) # sigmoid
    def fit(self, X, y):
        H = self._activation(X @ self.W + self.b)
        self.beta = np.linalg.pinv(H) @ y
    def predict(self, X):
        H = self._activation(X @ self.W + self.b)
        return H @ self.beta
# Train ELM
elm = ELM_Model(X_train_final.shape[1], 200)
elm.fit(X_train_final, y_train.values)
elm_pred = elm.predict(X_test_final).flatten()
```

RVFL:

```
def rvfl(X_train, X_test):
    W = np.random.randn(X_train.shape[1], 100)
    H_train = np.tanh(X_train @ W)
    H_test = np.tanh(X_test @ W)
    H_train = np.hstack([X_train, H_train])
    H_test = np.hstack([X_test, H_test])
    beta = np.linalg.pinv(H_train) @ y_train
    return H_test @ beta
rvfl_pred = rvfl(X_train_final, X_test_final)
rvfl_pred = rvfl_pred.flatten()
```

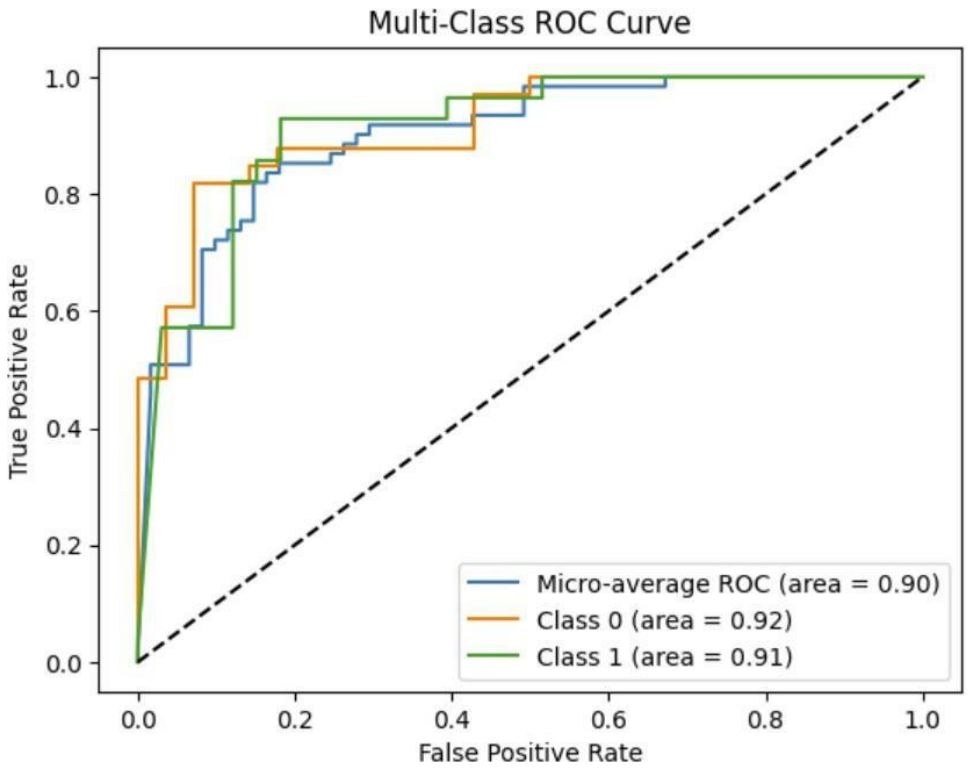


Fig7(i). ROC CURVE FOR HEART DISEASE

DEEP RVFL:

```
def deep_rvfl(X_train, X_test, y_train):
    # Convert y to numpy (CRITICAL FIX)
    y_train_np = y_train.values if hasattr(y_train, "values") else y_train
    H_train = X_train.copy()
    H_test = X_test.copy()
    for _ in range(3):
        W = np.random.randn(H_train.shape[1], 100)
        H_train = np.tanh(H_train @ W)
        H_test = np.tanh(H_test @ W)
    beta = np.linalg.pinv(H_train) @ y_train_np
    return H_test @ beta
```

GAN:

```
from tensorflow.keras import layers
```

```
latent_dim = 16
# Generator
generator = Sequential([
    Dense(32, activation='relu', input_dim=latent_dim),
    Dense(64, activation='relu'),
    Dense(X_train.shape[1], activation='linear')
])
# Discriminator
discriminator = Sequential([
    Dense(64, activation='relu', input_dim=X_train.shape[1]),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])
discriminator.compile(optimizer='adam', loss='binary_crossentropy')
# GAN
discriminator.trainable = False
gan_input = Input(shape=(latent_dim,))
fake_data = generator(gan_input)
gan_output = discriminator(fake_data)
gan = Model(gan_input, gan_output)
gan.compile(optimizer='adam', loss='binary_crossentropy')
# Training GAN
for epoch in range(200):
    noise = np.random.normal(0, 1, (X_train.shape[0], latent_dim))
    generated = generator.predict(noise, verbose=0)
    real_labels = np.ones((X_train.shape[0], 1))
    fake_labels = np.zeros((X_train.shape[0], 1))
    discriminator.trainable = True
    discriminator.train_on_batch(X_train, real_labels)
    discriminator.train_on_batch(generated, fake_labels)
    discriminator.trainable = False
    gan.train_on_batch(noise, real_labels)
# Generate synthetic samples
noise = np.random.normal(0, 1, (200, latent_dim))
```

```
gan_samples = generator.predict(noise)
# Add to training data
X_train = np.vstack([X_train, gan_samples])
y_train = np.hstack([y_train, np.random.choice(y_train, size=200)])
```

STACKING(DEEP RVFL+AUTO ENCODER):

```
#Ensure clean numpy arrays
X_train_final = np.asarray(X_train_final).astype(np.float32)
X_test_final = np.asarray(X_test_final).astype(np.float32)
y_train = np.asarray(y_train).astype(np.float32)
y_test = np.asarray(y_test).astype(np.float32)
print("Final shapes check:")
print(X_train_final.shape, y_train.shape)
#Split ONLY ONCE for stacking
from sklearn.model_selection import train_test_split
X_base, X_meta, y_base, y_meta = train_test_split(
    X_train_final, y_train, test_size=0.2, random_state=42
)
#Recreate models with correct input size
def create_dnn(input_dim):
    model = Sequential([
        Dense(128, activation='relu', input_shape=(input_dim,)),
        Dropout(0.3),
        Dense(64, activation='relu'),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model
dnn = create_dnn(X_base.shape[1])
rnn = Sequential([
    tf.keras.layers.SimpleRNN(32, activation='relu',
                              input_shape=(X_base.shape[1], 1)),
```

```
    Dense(1, activation='sigmoid')
])
rnn.compile(optimizer='adam',
            loss='binary_crossentropy',
            metrics=['accuracy'])
elm = ELM_Model(X_base.shape[1], 100)
#Train base models
dnn.fit(X_base, y_base, epochs=40, batch_size=16, verbose=0)
X_base_rnn = X_base.reshape((X_base.shape[0], X_base.shape[1], 1))
rnn.fit(X_base_rnn, y_base, epochs=30, batch_size=16, verbose=0)
elm.fit(X_base, y_base)
#Deep RVFL
deep_meta = deep_rvfl(X_base, X_meta, y_base)
#Create meta features
dnn_meta = dnn.predict(X_meta).flatten()
rnn_meta = rnn.predict(X_meta.reshape((X_meta.shape[0], X_meta.shape[1], 1))).flatten()
elm_meta = elm.predict(X_meta).flatten()
stack_meta = np.vstack([
    dnn_meta,
    rnn_meta,
    elm_meta,
    deep_meta
]).T
#Train meta model
meta_model = XGBClassifier(
    n_estimators=150,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8
)
meta_model.fit(stack_meta, y_meta)
#FINAL TEST
dnn_test = dnn.predict(X_test_final).flatten()
```

```
rnn_test = rnn.predict(X_test_final.reshape((X_test_final.shape[0], X_test_final.shape[1], 1))).flatten()
elm_test = elm.predict(X_test_final).flatten()
deep_test = deep_rvfl(X_train_final, X_test_final, y_train)
stack_test = np.vstack([
    dnn_test,
    rnn_test,
    elm_test,
    deep_test
]).T
final_pred = meta_model.predict(stack_test)
#FINAL ACCURACY
acc = accuracy_score(y_test, final_pred)
print("FINAL STACKING ACCURACY:", acc * 100)
```

```
FINAL STACKING ACCURACY: 90.1639344262295
```

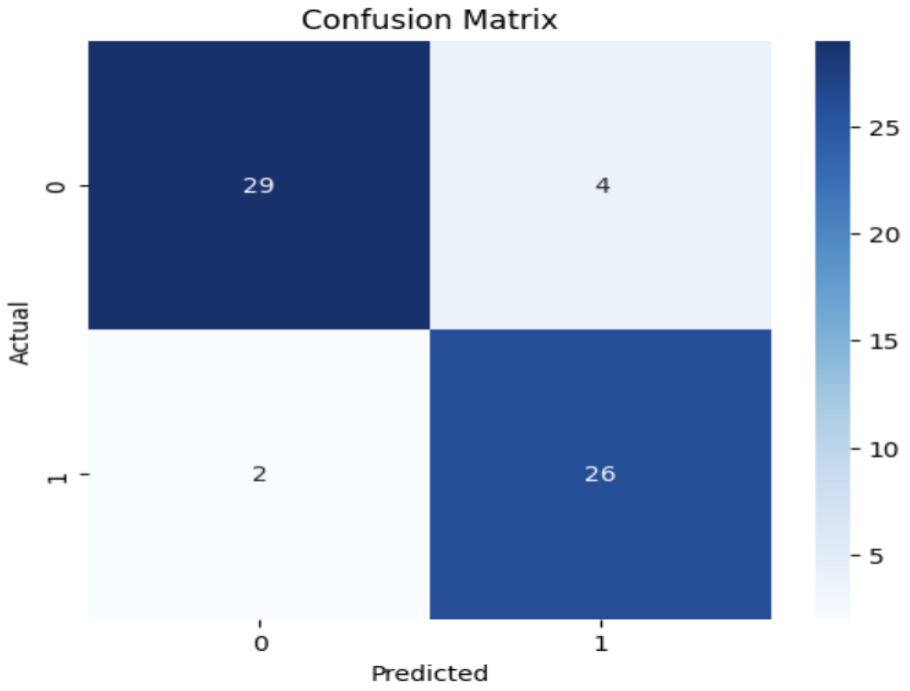


Fig7(j). CONFUSION MATRIX HEART DISEASE

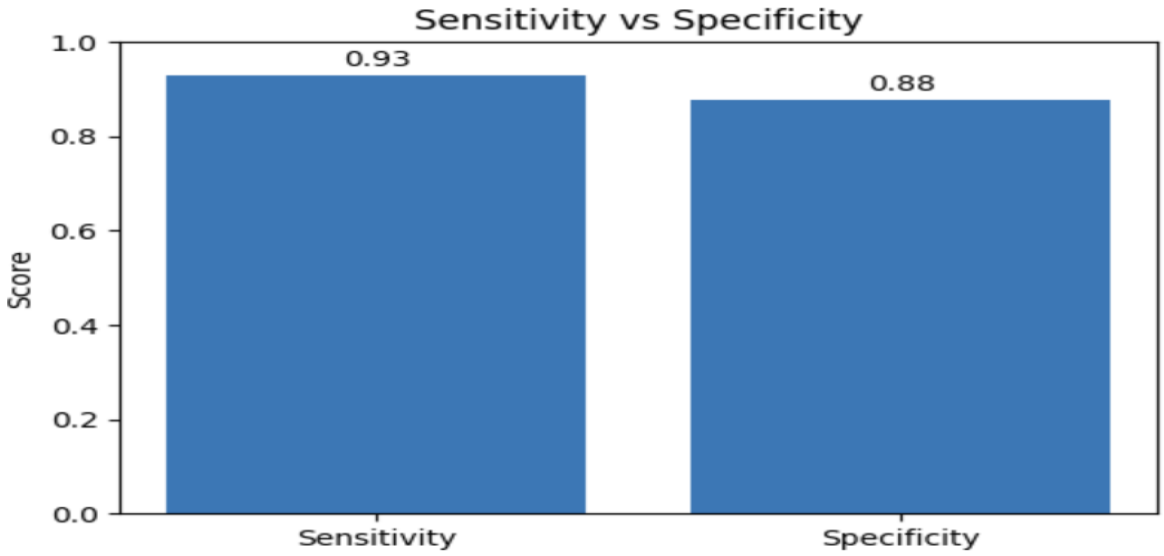


Fig7(k). SENSITIVITY VS SPECIFICITY FOR HEART DISEASE

LIME:

```
from lime.lime_tabular import LimeTabularExplainer
import numpy as np
```

```
explainer = LimeTabularExplainer(
    training_data=X_train_final,
    feature_names=[f"f{i}" for i in range(X_train_final.shape[1])],
    class_names=["No Disease", "Disease"],
    mode="classification"
)
#predict function
def predict_fn(data):
    probs = snn.predict(data)
    # Ensure shape (N,2)
    if probs.shape[1] == 1:
        probs = np.hstack([1 - probs, probs])
    return probs
# Explain one sample
exp = explainer.explain_instance(
    X_test_final[0],
    predict_fn,
    num_features=8
)
exp.show_in_notebook(show_table=True)
fig = exp.as_pyplot_figure()
plt.show()
```

Fig7(l). LIME FOR HEART DISEASE

SHAP:

```
import shap
background = X_train_final[np.random.choice(X_train_final.shape[0], 100, replace=False)]
KernelExplainer
explainer = shap.KernelExplainer(
    lambda x: snn.predict(x)[:, 1],
    background
)
# Compute SHAP values
shap_values = explainer.shap_values(X_test_final[:50])
#SUMMARY PLOT
shap.summary_plot(
    shap_values,
    X_test_final[:50],
    feature_names=[f"f{i}" for i in range(X_test_final.shape[1])]
)
#BAR IMPORTANCE
shap.summary_plot(
```

```
    shap_values,
    X_test_final[:50],
    plot_type="bar",
    feature_names=[f"f{i}" for i in range(X_test_final.shape[1])]
)
```

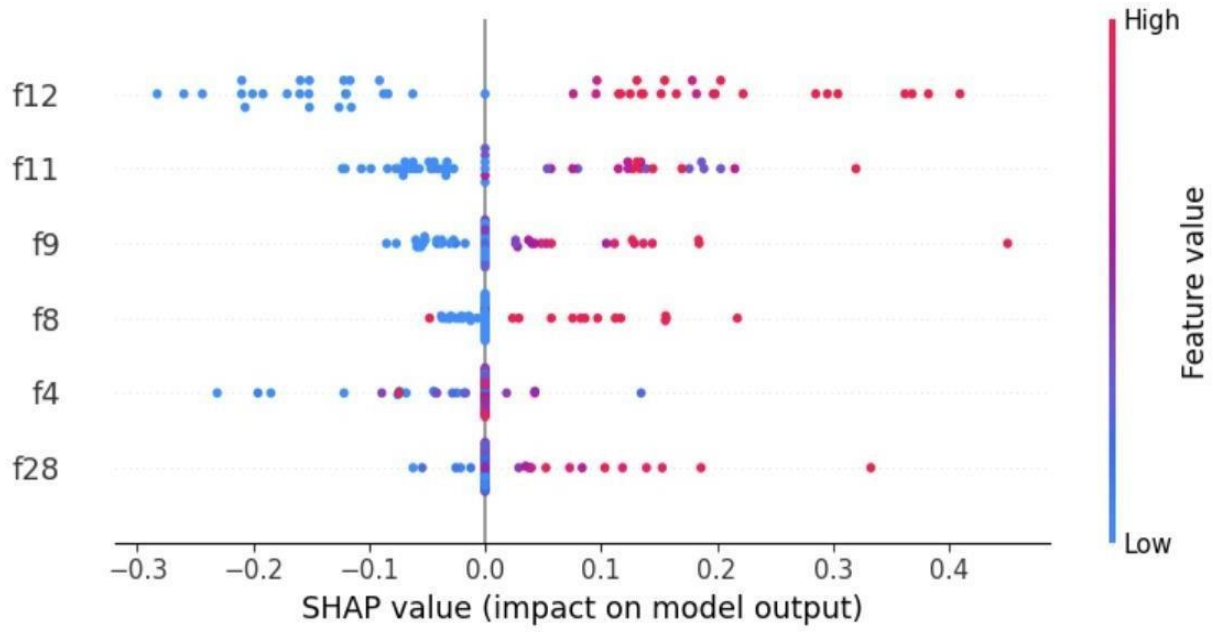


Fig7(m). SHAP FOR HEART DISEASE

FINAL MODEL COMPARISON TABLE

	Approach (Model)	Accuracy (%)	Precision	Recall	F1-Score	Training Time (s)
6	Stacking (Final Model)	90.16	0.8667	0.9286	0.8966	0.14
2	SNN	83.61	0.7500	0.9643	0.8438	6.14
0	DNN	80.33	0.7222	0.9286	0.8125	19.86
1	RNN	78.69	0.7586	0.7857	0.7719	10.26
3	ELM	73.77	0.6765	0.8214	0.7419	0.01
5	Deep RVFL	72.13	0.6667	0.7857	0.7213	0.02
4	RVFL	70.49	0.6389	0.8214	0.7188	0.02

Fig7(n). FINAL ACCURACY TABLE FOR HEART DISEASE

Aws Outputs:

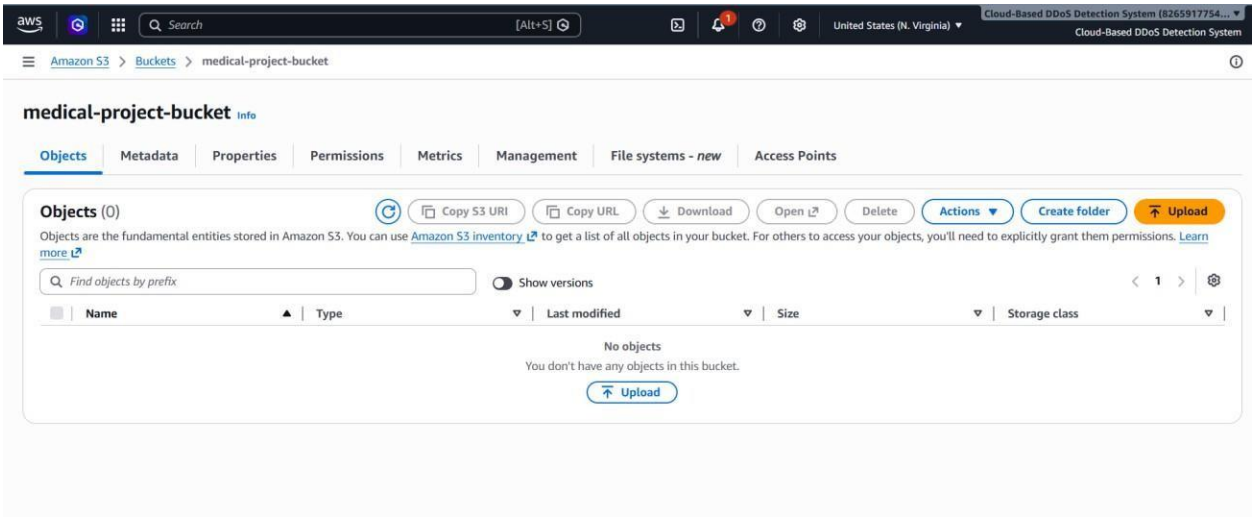


Fig7(o). BUCKET CREATION

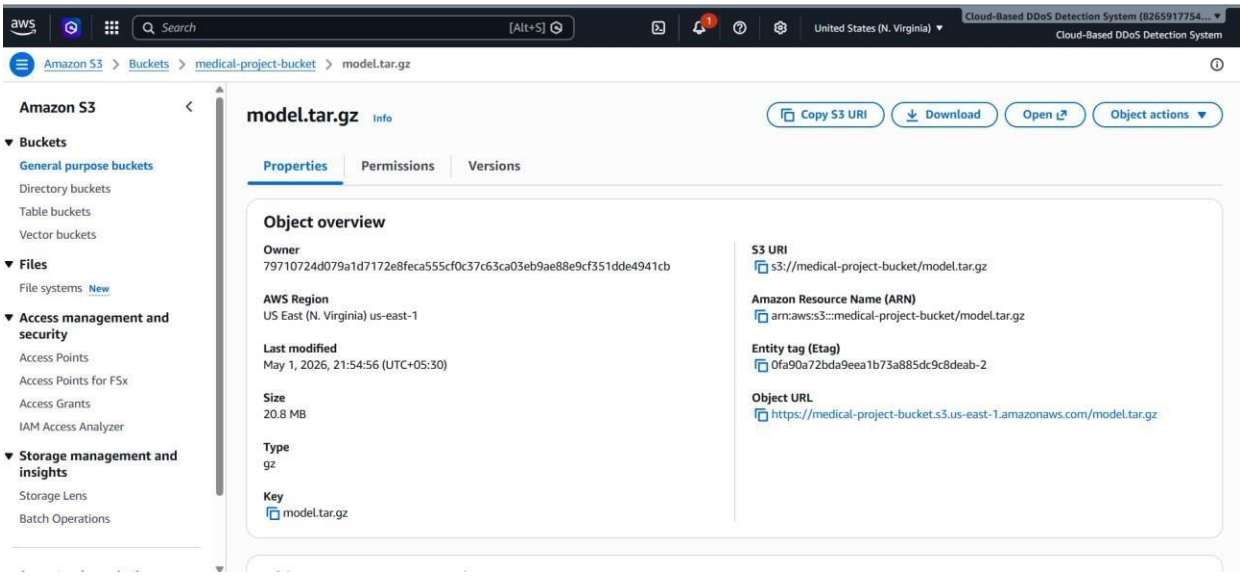


Fig7(p). FILES PRESENT IN BUCKET OVERVIEW

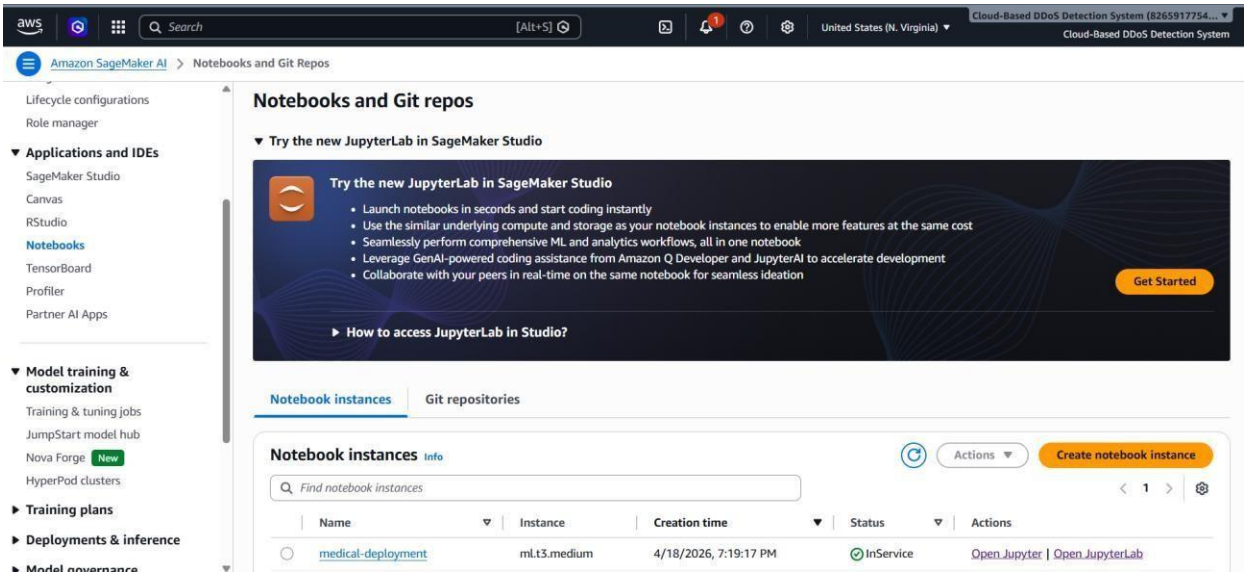


Fig7(q). SAGE MAKER INSTANCE CREATION

APPENDIX – BASE PAPER

Base Paper:

A. Sharma, S. Yadav, and M. Sharma, "Randomized Explainable Machine Learning Models for Efficient Medical Diagnosis," (base study referenced in this project).

Dataset Used:

Source: Kaggle

Name: Heart Disease Dataset, Genitourinary Cancer Dataset

Type: Structured medical dataset (CSV format)

Content: Contains patient clinical features such as age, medical history, test results, and diagnosis labels

Dataset Details:

Size: Varies across datasets (multiple medical records with labeled outputs)

Data Split:

Train: 80% of data

Test: 20% of data

Attributes:

Each record includes multiple medical features (e.g., age, blood pressure, cholesterol, etc.) along with a target label indicating presence or absence of disease.

This project utilizes publicly available medical datasets and does not involve direct human or animal experimentation. Hence, it does not require approvals from regulatory bodies such as Institutional Ethics Committee (IEC), Institutional Biosafety Committee (IBC), or similar authorities.